UNIVERSIDAD DEL QUINDIO
Programa de Ingeniería de Sistemas y Computación
PROGRAMACIÓN 3
PARCIAL II

1.0 (2.5) Punto 1: Observatorio Sismológico de la Universidad del Quindío

El Observatorio Sismológico de la Universidad del Quindío desea implementar una aplicación en Elixir para que los estudiantes consulten los movimientos sísmicos registrados en Colombia. Para el parcial, la aplicación trabajará con una copia local de los datos del Servicio Geológico Colombiano, distribuida en dos archivos. Cada vez que se solicite generar el reporte, la aplicación deberá leer ambos archivos para reflejar los cambios que se hayan hecho desde la última consulta.

El objetivo de la aplicación es **generar un reporte de los sismos por ciudad**, con el detalle de los movimientos registrados y un resumen de su magnitud y profundidad.

1. Archivos de entrada

Los archivos de entrada no tienen encabezado. Cada línea contiene un registro y los campos se separan mediante @.

Movimientos sísmicos — resources/movimientos.txt

código@idCiudad@horaInicio@horaFinal@magnitud@profundidad

Campo	Descripción
código	Identificador único del movimiento sísmico.
idCiudad	Identificador de la ciudad asociada al movimiento.
horaInicio	Hora de inicio del movimiento, en formato HH:MM:SS.
horaFinal	Hora de finalización del movimiento, en formato HH:MM:SS.
magnitud	Magnitud del movimiento, expresada como un número decimal.
profundidad	Profundidad del movimiento en kilómetros, expresada como un número mayor o igual a cero.

S001@C01@08:15:02@08:15:18@3.4@18.0
S002@C02@09:30:10@09:30:29@4.1@25.0
S003@C01@11:42:00@11:42:12@2.8@10.0

Ciudades — resources/ciudades.txt

idCiudad@nombre@@numeroHabitantes@

Campo	Descripción
idCiudad	Identificador único de la ciudad.
nombre	Nombre de la ciudad.
numeroHabitantes	Número de habitantes, expresado como un entero mayor que cero.

C01@Armenia@304314
C02@Calarcá@78684
C03@Circasia@30941

2. Funcionalidades requeridas

2.1. Leer los archivos de forma concurrente

Al solicitar un reporte, un **proceso coordinador** deberá iniciar dos procesos trabajadores. Uno leerá y procesará el archivo de ciudades y el otro leerá y procesará el archivo de movimientos sísmicos.

El coordinador deberá iniciar ambos trabajadores antes de esperar sus respuestas. Cada trabajador deberá enviar al coordinador un mensaje que identifique el archivo procesado y comunique el resultado de la lectura o el error ocurrido. El coordinador deberá aceptar las respuestas en cualquier orden y generar el reporte únicamente cuando tenga los resultados exitosos de ambos archivos.

2.2. Representar y validar los registros

La aplicación deberá representar las ciudades y los movimientos sísmicos mediante **dos structs diferentes**. Los campos numéricos deberán almacenarse con el tipo correspondiente para poder realizar las operaciones del reporte.

Las líneas vacías se ignorarán. Si una línea no tiene el formato indicado, contiene un campo obligatorio vacío o tiene un dato inválido, deberá excluirse del reporte y contabilizarse como **registro rechazado**. También se rechazará todo movimiento cuyo idCiudad no corresponda a una ciudad válida del catálogo.

Para este ejercicio, se garantiza que los códigos de movimiento y los identificadores de ciudad no se repiten. Una magnitud es válida si puede convertirse en un número decimal. Las dos horas se validarán como horas del día.

2.3. Generar el reporte por ciudad

La aplicación deberá mostrar **todas las ciudades válidas** del catálogo, incluso aquellas que no tengan movimientos asociados. Para cada ciudad deberá presentar:

- Nombre y número de habitantes.
- Listado de movimientos con código, hora de inicio, hora final, magnitud y profundidad.
- Cantidad total de movimientos registrados.
- Código y magnitud del movimiento de mayor magnitud.
- Profundidad promedio de los movimientos, con dos cifras decimales.

Si varios movimientos tienen la misma magnitud máxima, se mostrará el primero que aparezca en el archivo. Si una ciudad no tiene movimientos, el reporte indicará **«Sin movimientos registrados»**, mostrará un total de 0 y usará **«No aplica»** para la magnitud máxima y la profundidad promedio.

Al final se mostrará el total de registros rechazados, separado por archivo. Cada línea rechazada se contará una sola vez.

2.4. Guardar el reporte

El mismo reporte presentado en consola deberá guardarse en resources/reporte_sismos.txt. Cada generación exitosa reemplazará el contenido anterior. Si alguno de los dos archivos de entrada no se puede leer, la aplicación deberá indicar cuál produjo el error y conservar el reporte anterior. Si falla la escritura del reporte, deberá informar el error en consola.

3. Requisitos de implementación

- Utilizar spawn, send y receive para crear los procesos y comunicar sus resultados.
- Implementar **recursividad explícita** para recorrer los movimientos de cada ciudad y calcular, en un mismo recorrido, la cantidad de movimientos, el de mayor magnitud y la suma de profundidades. A partir de la suma y la cantidad se calculará el promedio.
- Separar la lectura de archivos, el cálculo del reporte y la coordinación de procesos en funciones con responsabilidades diferenciadas.

4. Resultado esperado con los datos de ejemplo

Ciudad	Movimientos	Movimiento de mayor magnitud	Profundidad promedio
Armenia	2	S001 · 3.4	14.00 km
Calarcá	1	S002 · 4.1	25.00 km
Circasia	0	No aplica	No aplica

La tabla resume los cálculos esperados. El reporte generado también deberá incluir el número de habitantes, el detalle de cada movimiento y el conteo de registros rechazados.

2.0 (2.5) Punto 2 : Consulta de reportes sísmicos entre nodos

El Observatorio Sismológico de la Universidad del Quindío desea que los estudiantes puedan consultar los reportes sísmicos desde un nodo de Elixir diferente al que almacena los datos. Para ello, se deberá implementar un **nodo cliente**, que envíe las consultas y muestre las respuestas, y un **nodo servidor**, que procese las solicitudes utilizando los archivos `resources/ciudades.txt` y `resources/movimientos.txt` del punto 1.

1. Consultas que deberá atender el servidor

Solicitud	Respuesta esperada
`{:reporte_ciudad, idCiudad}`	Reporte de la ciudad indicada. Deberá incluir el nombre, el número de habitantes, el detalle de sus movimientos, la cantidad total, el código y la magnitud del movimiento de mayor magnitud y la profundidad promedio con dos decimales.
`:reporte_general`	Reporte de todas las ciudades válidas, con el detalle y las estadísticas del punto 1 y el conteo de registros rechazados por archivo.
`:fin`	Confirmación `:fin` para indicar al cliente que ha terminado el envío de respuestas.

Si la ciudad existe y no tiene movimientos asociados, el reporte deberá indicar **«Sin movimientos registrados»**, mostrar un total de `0` y presentar **«No aplica»** para la magnitud máxima y la profundidad promedio. Si el identificador no corresponde a una ciudad válida, el servidor deberá responder **«La ciudad no está registrada»**.

2. Funcionamiento del nodo cliente

El cliente deberá registrar un proceso con el nombre `:servicio_respuesta` e intentar conectarse al nodo servidor mediante `Node.connect/1`. Si la conexión falla, deberá informar el error en consola y finalizar.

Una vez conectado, el cliente deberá enviar al proceso `:servicio_sismos` del nodo servidor, en este orden, las siguientes solicitudes.

1. El reporte de una ciudad con movimientos registrados.
2. El reporte de una ciudad sin movimientos registrados.
3. El reporte de una ciudad con un identificador inexistente.
4. El reporte general.
5. El mensaje `:fin`.

Cada mensaje deberá tener la forma `{destino_respuesta, solicitud}`, donde `destino_respuesta` identifica el proceso registrado y el nodo cliente. Después de enviar

las solicitudes, el cliente deberá recibir y mostrar cada respuesta en consola. La recepción deberá continuar de manera recursiva hasta recibir `:fin`.

3. Funcionamiento del nodo servidor

El servidor deberá registrar un proceso con el nombre `:servicio_sismos` y esperar solicitudes. Para cada consulta de reporte deberá leer concurrentemente las versiones actuales de ambos archivos, reutilizar los structs, las validaciones y los cálculos del punto 1, y enviar el resultado al destino indicado por el cliente. Si alguno de los archivos no puede leerse, deberá responder con un mensaje que identifique el archivo y continuar atendiendo solicitudes.

El servidor deberá responder en el mismo orden en que recibe las solicitudes de un cliente. Al recibir `:fin`, deberá enviar la confirmación después de las respuestas anteriores y volver a esperar solicitudes mediante una llamada recursiva.

4. Verificación

La comunicación entre nodos deberá realizarse con `send` y `receive`. Para verificar el funcionamiento, se ejecutará el cliente **dos veces consecutivas con el mismo servidor**. En cada ejecución, el cliente deberá mostrar las cuatro respuestas en el orden solicitado y finalizar al recibir `:fin`.

Entrega del parcial

Deberá entregar **el enlace a un único repositorio de GitHub** que contenga la solución completa de los dos puntos. El repositorio deberá ser accesible al momento de la revisión. Si es privado, el estudiante deberá otorgarle acceso antes de entregar el enlace.

1. Organización del repositorio

Cada punto deberá estar en una carpeta independiente. El `README.md` deberá estar en la raíz del repositorio.

```
None
parcial-2-elixir/
├── README.md
├── punto-1/
│   ├── Archivos .exs del reporte local
│   └── resources/
│       ├── ciudades.txt
│       └── movimientos.txt
└── punto-2/
    ├── nodo-cliente/
```

```
│   └── Archivos .exs del cliente
└── nodo-servidor/
    ├── Archivos .exs del servidor
    └── resources/
        ├── ciudades.txt
        └── movimientos.txt
```

Se deberán incluir el código fuente y los archivos de entrada necesarios para probar ambos puntos. Los datos de prueba deberán incluir, como mínimo, **una ciudad con movimientos y una ciudad sin movimientos**.

2. Instrucciones de ejecución en el `README.md`

El `README.md` deberá describir cómo ejecutar cada punto. Para cada comando, deberá indicar la carpeta desde la que se ejecuta.

Punto	Información que deberá incluir el `README.md`
Punto 1 — Reporte local	Comando para ejecutar el programa, ubicación de los archivos de entrada y ruta del reporte que se genera, `resources/reporte_sismos.txt`.
Punto 2 — Consulta entre nodos	Instrucciones para abrir dos terminales, iniciar primero el servidor y luego el cliente. Deberá incluir los comandos completos, los nombres de los nodos y la forma de configurar la misma cookie en ambos.
Verificación del punto 2	Identificadores usados para probar una ciudad con movimientos, una ciudad sin movimientos y una ciudad inexistente. Deberá indicar cómo ejecutar el cliente por segunda vez con el mismo servidor para comprobar que sigue atendiendo solicitudes.

Reflexión sobre el uso de inteligencia artificial

Si utilizaron IA, respondan con evidencias concretas; si no, expliquen cómo investigaron y resolvieron sus dudas.
1. Herramientas y agentes. Indiquen qué asistentes o extensiones utilizaron y en qué tareas.
2. Caso decisivo. Describan una interacción específica, la respuesta obtenida y cómo la verificaron o adaptaron.
3. Idea diferente de la clase. Analicen una sugerencia distinta de las vistas en clase; su aporte, si la implementaron y lo aprendido.
4. Conclusiones. Identifiquen qué comprenden por sí solos, qué les cuesta todavía y qué cambiarían en un siguiente proyecto.

En la sustentación se podrán pedir explicaciones, modificaciones, nuevos requisitos a cualquier parte del código, incluso si fue escrito con ayuda de IA.